1 Computation, Ambiguity, and Formal Languages

Every time you type a query, speak to a voice assistant, or hit compile, something is deciding whether your input counts as "acceptable." Strip away the specifics, and each is answering the same question: *does this input belong to a particular set of valid inputs?* Answering it *reliably* means making "valid" mathematically precise – and that is what these notes chase: why natural language fails at this, how a misplaced `else` in C++ makes the failure concrete, and how formal languages fix it with three small pieces. We stop right where the machinery is in place but the big question – how to pin down a computational problem with it – is still open; answering that is the rest of the course.

1.1 What Does It Mean to Compute?

We have not yet said what computing *means*. "Doing arithmetic fast" is too narrow – a spell-checker, a chess engine, a search engine all compute, and none is doing arithmetic. A great many of these tasks, on inspection, turn out to be testing membership: is this word in the dictionary? Is this move legal? Does this page match the query? That is not the whole of computing – plenty of tasks produce a number, a string, or an object rather than a yes/no – but it is a productive enough pattern that this course builds its foundations around it, so we follow that thread first.

A question many algorithms are really answering

"Does this input belong to a particular set of valid inputs?" Not every computation reduces to this cleanly, but a great deal of what this course studies does. To answer it *reliably* – the same way, every time, for every input – we must first make "valid" mathematically precise. Vague or intuitive notions of validity are fine for a human reader, but a machine cannot execute a vibe.

This is not a new question – it is at least a century old, and computer science is arguably just its latest chapter.

Where this question has been asked before

In 1900, David Hilbert asked whether mathematics could be settled mechanically – his slogan was "Wir müssen wissen, wir werden wissen" ("We must know, we will know"). In 1931, in the same city, days before Hilbert repeated that slogan in his retirement address, Kurt Gödel showed that sufficiently expressive formal systems cannot prove all true statements. Church and Turing sharpened that wound five years later: in 1936, independently, they showed that no general mechanical procedure can decide all mathematical truths, and in the process handed mathematics a precise, mechanical notion of *algorithm* – the Turing machine and the λ-calculus. Every model this course studies descends from that 1936 answer.

So, within that lens, "what does it mean to compute" becomes "what does it mean to decide membership in a set, mechanically and reliably." We need "set of valid inputs" to stop being a vague phrase and become something precise.

Exercises: What Does It Mean to Compute?

1. For each of the following, phrase it as a "does this input belong to a particular set of valid inputs" question: (a) a spam filter, (b) an autocomplete suggestion feature, (c) a login system checking a password.

2. Can you think of a real system you use daily whose underlying decision *cannot* be phrased this way? (Hint: more tasks reduce to membership than they first appear to – but not all. Think about what changes when a task's output is a specific string, number, or object, rather than a yes/no.)

1.2 Natural Language Is Ambiguous

The obvious tool for describing "the set of valid inputs" is natural language – English, here. It is powerful precisely because it carries nuance no formal notation matches. But that power is fatal once a machine is listening: natural language is **ambiguous** by design, and human listeners resolve that ambiguity using context a machine does not have.

Ambiguous English

"I saw the man with the telescope."

Did you see the man *using* a telescope, or one who *happened to be carrying* one? Both readings are grammatical, and nothing in the sentence picks one – two people might disagree. A machine, given only the words, has no principled way to choose either; it needs to resolve every input the *same way, every time*, and "read the room" is not an algorithm.

Exercises: Ambiguity in Natural Language

1. Find (or recall) one more English sentence with two genuinely different readings, and write out both readings explicitly. (Hint: sentences with the word "or," or with a prepositional phrase like "with," "near," or "on," are a good place to hunt.)

2. A menu sign reads: "Children's portions half price." List two different rules a cashier could honestly follow based on this sentence.

1.3 The Cost of Ambiguity: The Dangling-Else Problem

Surely a language built to be precise does not have this problem? Not quite – consider this ordinary C++:

The dangling-else problem

```
if (x)
    if (y)
        a();
    else
        b();
```

Does the `else` belong to the inner `if (y)`, or to the outer `if (x)`?

The indentation suggests `else` pairs with `if (x)` – but indentation is not part of the grammar; the compiler ignores it. Stripped to tokens, C++'s `if...else` is genuinely ambiguous between two parse trees, exactly like the telescope sentence. Get it wrong and `b()` silently runs under an unintended condition – no error, just a bug that surfaces days later.

A compiler cannot shrug and say "it depends on context" – it must resolve every input the *same way, every time*. So C and C++ legislate the ambiguity away: **`else` binds to the nearest unmatched `if`**. Under that rule, our example's `else` belongs to `if (y)`, the inner one – despite the indentation.

To think!!

The "nearest unmatched `if`" rule fixes the ambiguity, but it does so by *picking a convention*, not by proving one reading was secretly more correct than the other. If you genuinely wanted the `else` to belong to the outer `if (x)` instead, how would you rewrite the code to force that, given that the language's default rule works against you?

A preview: the same ambiguity as a grammar

The naive grammar for if-else is ambiguous: written down formally (Lecture 3 gives grammars their tuple), something like Stmt $\rightarrow$ `if`(Expr) Stmt | `if`(Expr) Stmt `else` Stmt | ... lets the very token string above be built by two different parse trees. The "nearest unmatched `if`" rule is one standard fix; we meet the general problem – and other fixes – once grammars are on the table.

Any language whose rules are stated informally is a language whose meaning can be argued about. If a compiler needs one unambiguous answer for every input, English-style description is not good enough as a *specification* – even for a programming language. We need a language defined so precisely that "is this string valid, and what does it mean" has exactly one answer, checkable by a machine. That is what formal languages are for.

Exercises: The Dangling-Else Problem

1. Using the "`else` binds to the nearest unmatched `if`" rule, determine which `if` the `else` binds to:

```
if (a)
    if (b)
        f();
if (c)
    g();
else
    h();
```

2. Rewrite the original dangling-`else` example from this section using curly braces {} so that the `else` unambiguously belongs to the outer `if (x)` instead. (Hint: an empty block {} can stand in for the inner `if`'s missing body.)

1.4 From Ambiguity to Formal Languages

The fix, in both cases, is the same: replace "sounds about right" with an **exact, checkable rule**. A **formal language** is exactly that – a precise object built from primitive pieces, leaving no room for two people (or two compilers) to disagree about membership. Building it needs exactly three ingredients, stacked one on the other: an alphabet, strings over it, and a language as a chosen set of those strings.

1.5 Building Block 1: Alphabet

The most primitive ingredient: a finite collection of symbols we are allowed to use, before any combining.

Alphabet

An **alphabet** Σ is simply a finite set of symbols.

Examples

- $\Sigma = \{0, 1\}$ – the binary alphabet.
- $\Sigma = \{a, b, \ldots, z\}$ – lowercase English letters.
- $\Sigma = \{\texttt{if}, \texttt{else}, (,), \ldots\}$ – the tokens of a programming language (yes, the very `if` and `else` from Section 1.3 are themselves symbols of C++'s alphabet).

"Finite" does real work here – it is what lets us build everything else (strings, languages) out of finitely describable pieces, even though those turn out infinite.

Exercises: Alphabets

1. Is $\Sigma = \{\text{red}, \text{green}, \text{blue}\}$ a valid alphabet? Justify your answer using the definition above.
2. Write down an alphabet suitable for describing decimal (base-10) numbers.
3. Why do you think the definition insists Σ be *finite*? (Hint: think about what it would mean to write down, or check membership in, an infinite alphabet.)

1.6 Building Block 2: Strings

A loose symbol alone says little – as little as a single letter says in English. The next building block is putting symbols in a sequence.

String

A **string** is a finite sequence of symbols from Σ.

Over $\Sigma = \{0, 1\}$

- 0, 1, 01, 101, 0110 are all strings over Σ.
- The **empty string** ε – the sequence with zero symbols – is also a string, easy to forget precisely because it looks like nothing.
- $|w|$ denotes the **length** of a string w; for instance $|0110| = 4$ and $|\varepsilon| = 0$.

A quick myth-buster: a string is a *sequence*, not a set – order matters, symbols repeat freely. 011 and 101 share the same symbols but are different strings. Compare the alphabet $\Sigma = \{0, 1\}$ (Section 1.5) itself: as a *set*, it cannot repeat elements and ignores order – $\{0, 1\} = \{1, 0\}$. That set-versus-sequence distinction is what separates an alphabet from a string.

Exercises: Strings

1. Let $\Sigma = \{0, 1\}$. Is 011 a string over Σ? Is $\{0, 1\}$ a string over Σ? Explain the difference.
2. Compute $|w|$ for $w = 10010$, and separately for $w = \varepsilon$.
3. True or false, with justification: "a symbol may never repeat within a string."

1.7 Building Block 3: Languages

We have an alphabet and strings built from it. The final block: which strings, out of all possible ones, do we actually care about – exactly the "set of valid inputs" from Section 1.1, made precise.

Language

A **language** L is a set of strings over Σ, i.e. $L \subseteq \Sigma^*$, where Σ^* denotes the set of *all* strings that can be built from Σ.

Over $\Sigma = \{0, 1\}$

Let $L = \{w : w \text{ is the binary encoding of an even number}\}$.

- $0, 10, 100, 110 \in L$.
- $1, 11, 101 \notin L$.

Three simple ingredients – symbols, sequences of them, and a chosen subset of those sequences – and we have an object with no "it depends on context" left in it. For any w and L, "is $w \in L$?" has exactly one right answer, checkable by anyone who agrees on Σ, strings, and L. This is the same ladder that will let us finally say, precisely, what the dangling-`else` rule (Section 1.3) means, and what a valid C++ program even is.

Exercises: Languages

1. Let $\Sigma = \{0, 1\}$ and $L = \{w : w \text{ is the binary encoding of an odd number}\}$. Which of 0, 1, 101, 110, 1111, 1000 belong to L? (Hint: a binary string represents an odd number exactly when it ends in 1.)
2. Using the same Σ, describe in words a language L' that contains ε.
3. Compare your language from the previous exercise with the even-number language L from this section's worked example – can a single string belong to both? Can a string belong to neither?

1.8 So How Do We Actually Define a Computational Problem?

Stack the three blocks: symbols sequence into strings, and a chosen set of strings is a language. That ladder settles, with no ambiguity, whether a string belongs to a language – which already answers the question we have been circling since Section 1.1, at least for the slice of computing this course is built around: **asserting membership of a string in a language**, and that act is what we call a **decision problem**.

Decision problem

Given a language $L \subseteq \Sigma^*$ and a string $w \in \Sigma^*$, the question "is $w \in L$?" is a **decision problem**. Asserting membership of a string in a language is what we call deciding it.

But notice what we have *not* done: we defined L above by describing it in a sentence – fine for us as readers, but exactly the trap we started trying to escape. A sentence is not a machine-checkable rule, and most languages we care about are infinite, so we cannot just list and check. So the question this document ends on, deliberately unanswered: **given only an**

alphabet and a language as a set of strings, how do we *finitely* and *mechanically* specify which strings are in L, so that membership can be decided rather than just asserted?

That is precisely where the rest of this course begins.

2 References

1. (SIPSER) Michael Sipser, *Introduction to the Theory of Computation*, 3rd ed., Cengage Learning, 2012.

2. (HMU) John E. Hopcroft, Rajeev Motwani, Jeffrey D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3rd ed., Pearson, 2006.

3 Further Readings

1. (HMU) §1.5 "The Central Concepts of Automata Theory" (1.5.1 Alphabets, 1.5.2 Strings, 1.5.3 Languages, 1.5.4 Problems) – the same three building blocks from this document, plus HMU's own treatment of how a language encodes a problem.

2. (SIPSER) §0.2 "Mathematical Notions and Terminology," under *Strings and Languages* – Sipser's version of the alphabet/string/language definitions built here.

3. (SIPSER) §3.3 "The Definition of Algorithm" – the history behind Section 1.1's question: Hilbert's Entscheidungsproblem, and how Church and Turing resolved it.

4 Disclaimer

The above document is intended to be a reading material/class note for an undergraduate course on Theory of Computation. This first draft may contain errors as it has not been reviewed. Feel free to let me know the gaps, errors, and typos you encounter in the document.

1 History, the Kleene Star, and Closure Properties

We left off on an open question: given Σ and a language L, how do we *finitely* specify which strings are in L? First, a quick pointer to how old that question really is. Then we build Σ^* precisely, using the **Kleene star** construction; generalise it to any language L; and define exactly what it means for languages to be **closed** under an operation.

1.1 A Hundred and Twenty Years of the Same Question

History deserves an essay, not a subsection – so go read one. Paul Ford's "What Is Code?" (Bloomberg Businessweek, 2015) is a long, famously readable walk through what code and computation are and how we got here; it was Bloomberg.com's most-read page the year it ran. Here, just the timeline from Lecture 1, to anchor the dates.

Timeline: the same question, six different eras

Year	Event
1900	Hilbert's Problems – is mathematics fully mechanisable?
1931	Gödel's Incompleteness Theorems – the limits of formal systems
1936	The Turing Machine & the λ-calculus (Turing, Church, independently)
1956	The Chomsky Hierarchy – grammars classify languages
1982	Feynman asks: can a classical computer simulate quantum physics?
1985	Deutsch's Quantum Turing Machine – a new model of computation
1994	Shor's Algorithm – quantum factoring
2019	"Quantum Supremacy" – Google's Sycamore processor

Every row is the same question in a new era: *what can this model compute, and how do we know?* The quantum half of the timeline gets analysed with the same alphabets/strings/languages toolkit we build below – every model this course meets is one more answer to a century-old question.

Exercises: A Hundred and Twenty Years of the Same Question

1. Read the linked article (or at least its opening section) and, in one or two sentences, note one thing it says about the history of computation that was not in the timeline above.

2. Of the eight events in the timeline, which one do you think most directly makes finite automata (the machines we meet next in this course) possible? Justify your choice.

1.2 From Strings to Σ^*: The Alphabet's Own Closure

Last time we used Σ^*, "the set of all strings over Σ," without pinning down what it contains. Σ^* is infinite, so we cannot list it – we need a *rule* that generates every string instead. The natural rule: start from nothing, and keep appending one symbol at a time.

Recursive definition of Σ^*

- **Base case**: $\varepsilon \in \Sigma^*$.
- **Inductive case**: if $w \in \Sigma^*$ and $a \in \Sigma$, then $wa \in \Sigma^*$.
- Σ^* is the *smallest* set closed under these two rules – nothing is thrown in that these two rules do not force in.

This is exactly "start empty, repeat, collect everything reachable" – a pattern with a name, **closure**. Σ^* is the closure of $\Sigma \cup \{\varepsilon\}$ under appending a symbol; Section 1.3 makes this precise for a general language.

A note on concatenation

The inductive case above generalises to **concatenation**: for $u, v \in \Sigma^*$, $u \cdot v$ (written uv) is u followed by v, defined recursively by

$$\varepsilon \cdot v = v, \qquad (wa) \cdot v = w\,(a \cdot v) \ \text{ for } a \in \Sigma.$$

Three facts follow immediately: ε is the **identity** ($\varepsilon w = w\varepsilon = w$); concatenation is **associative** ($(uv)\,w = u\,(vw)$) but **not commutative** ($ab \neq ba$); and **length adds**, $|uv| = |u| + |v|$.

Worked example: Σ^* by length

Fix $\Sigma = \{a, b\}$. Building Σ^* level by level, by length, gives:

- Length 0: ε (1 string)
- Length 1: $a,\ b$ (2 strings)
- Length 2: $aa,\ ab,\ ba,\ bb$ (4 strings)

In general there are $|\Sigma|^n = 2^n$ strings of length n; Σ^* is the union of all levels. Concatenation: $u = ab,\ v = ba \implies uv = abba$, and $|uv| = 4 = 2 + 2 = |u| + |v|$.

Exercises: From Strings to Σ^*

1. Let $\Sigma = \{0, 1\}$. List every string in Σ^* of length ≤ 2, and check your count against $2^0 + 2^1 + 2^2$.
2. Let $u = 101$, $v = 010$. Compute uv and vu. Are they equal? What does this tell you about concatenation in general? (Hint: you already know the answer in words – write out the strings and see it directly.)
3. Using the recursive definition of concatenation, justify why $|uv| = |u| + |v|$ for all $u, v \in \Sigma^*$. (Hint: induct on the length of v, using the two defining equations.)

1.3 The Kleene Star of a Language

The same closure idea works one level up: instead of repeating a symbol, repeat an entire language L. For $L = \{ab\}$, "repeat L zero or more times" gives $\varepsilon,\ ab,\ abab, \ldots$ – this is the **Kleene star** of L, written L^*: the same construction as Section 1.2, applied to languages instead of symbols.

Kleene star and positive closure

For a language $L \subseteq \Sigma^*$, define L's powers inductively:

$$L^0 = \{\varepsilon\}, \qquad L^{i+1} = L^i L \ \text{ for } i \geq 0.$$

Then

$$L^* = \bigcup_{i=0}^{\infty} L^i \quad \text{(Kleene star)}, \qquad L^+ = \bigcup_{i=1}^{\infty} L^i = L\,L^* \quad \text{(positive closure)}.$$

Since $\varepsilon \in L^0 \subseteq L^*$ always, $L^* = L^+ \cup \{\varepsilon\}$ – the star is just the positive closure with ε thrown back in.

Kleene, and where the name actually comes from

The star is named after Stephen Cole Kleene – who introduced it not for programming languages (which barely existed yet) but in a 1951 RAND Corporation report on *McCulloch–Pitts neurons*, an idealised model of a biological neuron. That report stayed unpublished for five years, finally appearing in 1956 in the collection *Automata Studies* – the 1956 date you'll see attached to it in the References below. Formal language theory has one foot in neuroscience.

This also pins down what a **language** is: any subset of Σ^*, $L \subseteq \Sigma^*$ – nothing more. "Deciding" L (Lecture 1's decision-problem question) is just testing membership in whichever subset L is.

Worked example: powers of $L = \{a, b\}$

Let $\Sigma = \{a, b\}$ and $L = \{a, b\}$ – that is, $L = \Sigma$ itself, viewed as a language.

- $L^0 = \{\varepsilon\}$
- $L^1 = \{a, b\}$
- $L^2 = \{aa, ab, ba, bb\}$
- $L^* = \{\varepsilon, a, b, aa, ab, ba, bb, \dots\}$ – every string over Σ, i.e. $L^* = \Sigma^*$.

Σ^* (Section 1.2) is exactly this construction with $L = \Sigma$ – we were computing a Kleene star all along.

Notation at a glance

Symbol	Meaning
Σ^*	the set of *all* strings over Σ
$u \cdot v$ (or uv)	concatenation of strings u and v
L^i	L concatenated with itself i times; $L^0 = \{\varepsilon\}$
L^*	Kleene star – L repeated zero or more times
L^+	positive closure – L repeated one or more times

Exercises: The Kleene Star of a Language

1. Let $L = \{01\}$. Write out L^0, L^1, L^2, and the first four strings of L^* in increasing order of length.
2. Is $\varepsilon \in L^+$ for $L = \{01\}$ above? Justify your answer directly from the definition $L^+ = \bigcup_{i \geq 1} L^i$, without just citing the identity $L^* = L^+ \cup \{\varepsilon\}$.
3. Compare: for $L = \{a, b\}$ from the worked example, we found $L^* = \Sigma^*$. Is it true in general that $L^* = \Sigma^*$ for *any* language $L \subseteq \Sigma^*$? (Hint: try $L = \{aa\}$ over $\Sigma = \{a, b\}$ and see what strings you can, and cannot, reach.)

1.4 Closure Properties on Languages

We now have two ways to build languages from languages: concatenation and Kleene star. Languages are sets, so ordinary set operations apply too. The word tying all of this together is **closure** – though, as the box below makes precise, that one word actually covers two related but different ideas, worth pulling apart before they get used side by side.

Closure of a set under an operation

A set S is **closed** under an operation if applying the operation to element(s) of S never produces a result outside S. The **closure** of a set S under an operation is the *smallest* superset of S that *is* closed under it – keep applying the operation to whatever you have, and collect everything reachable.

L^* is exactly the closure of $L \cup \{\varepsilon\}$ under concatenation, and Σ^* is the closure of Σ under the same operation – Kleene star *is* closure, just for concatenation, in the sense the box above defines.

Two ideas share the word "closure" – keep them apart

- The **closure operator** (the box above) starts from *one* set and *builds* a new, possibly larger one. L^* is the closure operator applied to L under concatenation – it hands you back a set you did not have before.
- A **closure property** is a yes/no fact about a whole *class* of sets: does applying an operation to members of the class always land back inside the class? "The regular languages are closed under union" builds nothing new – it just asserts that $L_1 \cup L_2$ is already regular whenever L_1, L_2 are.

The rest of this subsection – despite its title – is mostly about the second sense. We are not building new sets; we are asking whether a class of languages, applied to an operation, already contains what it needs to.

The same closure-*property* question applies to other operations on $L, L_1, L_2 \subseteq \Sigma^*$:

The language operations

- $L_1 \cup L_2,\ \ L_1 \cap L_2$ (ordinary set union / intersection)
- $\overline{L} = \Sigma^* \setminus L$ (complement, relative to Σ^*)
- $L_1 L_2 = \{uv : u \in L_1,\ v \in L_2\}$ (concatenation of languages)
- $L^R = \{w^R : w \in L\}$, where w^R reverses w (reversal)
- $L^*,\ L^+$ (Kleene star / positive closure, Section 1.3)

Each operation takes subsets of Σ^* and returns another subset of Σ^*. This is the closure property that matters going forward: every language class we build (regular, context-free, ...) needs to stay closed under these, or the algebra breaks the moment two languages from a class combine into something outside it.

To think!!

Σ^* is closed under concatenation. But is an *arbitrary* $L \subseteq \Sigma^*$? Try $L = \{a\}$ over $\Sigma = \{a, b\}$: is $aa \in L$? If not, what does that tell you about why we needed the Kleene star construction at all?

Worked example: concatenating two closures

Let $\Sigma = \{a, b\}$, $L_1 = \{a\}^* = \{\varepsilon, a, aa, \dots\}$, and $L_2 = \{b\}^* = \{\varepsilon, b, bb, \dots\}$. Their concatenation is

$$L_1 L_2 = \{a^i b^j : i, j \geq 0\} = \{\varepsilon,\ a,\ b,\ aa,\ ab,\ bb,\ aab, \dots\}.$$

Every string in $L_1 L_2$ is some number of a's (possibly zero) followed by some number of b's (possibly zero) – never a b before an a, since L_1 always contributes the first piece. This is how the operations describe an infinite language compactly, without listing it.

Exercises: Closure Properties on Languages

1. Let $L = \{a, b\}$ (from Section 1.3's worked example). Compute L^2 directly from the definition $L^2 = LL$, and check it agrees with what you wrote down there.
2. Is the set of *finite* languages over $\Sigma = \{a, b\}$ closed under union? Is it closed under Kleene star? (Hint: what does L^* look like whenever $L \neq \emptyset$ and $L \neq \{\varepsilon\}$?)
3. Using $L_1 = \{a\}^*$ and $L_2 = \{b\}^*$ from the worked example above, compute $L_2 L_1$ (note the order is swapped) and compare it to $L_1 L_2$. Are they the same language? What does this tell you about whether concatenation of *languages* is commutative?

2 References

1. (SIPSER) Michael Sipser, *Introduction to the Theory of Computation*, 3rd ed., Cengage Learning, 2012.

2. (HMU) John E. Hopcroft, Rajeev Motwani, Jeffrey D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3rd ed., Pearson, 2006.

3. (KLEENE) Stephen C. Kleene, "Representation of Events in Nerve Nets and Finite Automata," in *Automata Studies*, Princeton University Press, 1956, pp. 3–41.

4. Paul Ford, "What Is Code?", Bloomberg Businessweek, June 2015.

3 Further Readings

1. (SIPSER) §3.3 "The Definition of Algorithm" – the same historical grounding as this document's timeline: Hilbert's Entscheidungsproblem, and how Church and Turing resolved it.

2. (HMU) §3.1 "Regular Expressions" (3.1.1 The Operators of Regular Expressions) – the star operator as it appears in regex syntax, one level above the abstract Kleene closure built in Section 1.3.

3. (HMU) §4.2 "Closure Properties of Regular Languages" (4.2.1 Closure of Regular Languages Under Boolean Operations, 4.2.2 Reversal) – the same union/intersection/ complement/reversal operations from this document's closing subsection, proved specifically for regular languages.

4 Disclaimer

The above document is intended to be a reading material/class note for an undergraduate course on Theory of Computation. This first draft may contain errors as it has not been reviewed. Feel free to let me know the gaps, errors, and typos you encounter in the document.

1 Recognizers, Regular Expressions, and Grammars

Lecture 1 ended on an open question: given only Σ and a language as a subset of Σ^*, how do we *finitely* specify which strings are in it? Lecture 2 built the algebra – Σ^*, concatenation, Kleene star, closure – that any answer would need. Now we answer it: there are exactly two dual ways to specify a language, **recognizers** and **generators**. We define both, then build out the generator side with two concrete tools: **regular expressions**, and **grammars** – stopping right at a grammar's production rules, before derivations.

1.1 The Specification Problem: Two Dual Answers

A language L can be pinned down in exactly two dual ways: build a machine that reads a string and answers yes or no, or build a device that writes out exactly the strings of L, without reading any input at all.

Recognizers and generators

- A **recognizer** (an **automaton**) is a machine M that takes a string w as input and outputs ACCEPT or REJECT. It defines the language $L(M) = \{w : M \text{ accepts } w\}$.
- A **generator** (a **grammar** or **regular expression**) is a device G that produces strings by rule, with no input to read. It defines the language $L(G) = \{w : G \text{ can generate } w\}$.

This course studies both halves. The rest of this document builds the generator side.

1.2 Regular Expressions: A Finite Notation for $L(R)$

Recall Lecture 2's algebra: concatenation, union, and Kleene star build new languages out of old ones. A **regular expression** is a finite string of symbols that names one specific language, built out of exactly those three operations, recursively.

Recursive definition of a regular expression over Σ

Base cases:

- $\emptyset$ is a regular expression, $L(\emptyset) = \emptyset$.
- ε is a regular expression, $L(\varepsilon) = \{\varepsilon\}$.
- For each $a \in \Sigma$, a is a regular expression, $L(a) = \{a\}$.

Inductive cases: if R_1, R_2 are regular expressions, so are

- $(R_1|R_2)$, with $L(R_1|R_2) = L(R_1) \cup L(R_2)$;
- (R_1R_2), with $L(R_1R_2) = L(R_1)L(R_2)$;
- $(R_1)^*$, with $L((R_1)^*) = L(R_1)^*$.

Nothing else is a regular expression.

Every piece of that definition already has a name from Lecture 2 – union, concatenation, Kleene star – just written as compact linear notation instead of set-builder notation. A regular expression *is* a recipe for building a language the same way we built L_1L_2 and L^* before, except now the recipe itself is a string you can write on one line.

grep, and where the notation actually earns its keep

Ken Thompson built regular-expression matching into the `QED` and `ed` text editors in the late 1960s, using exactly Kleene's 1951 closure notation (Lecture 2). The Unix command `grep` gets its name from the `ed` command `g/re/p` – "globally search for a regular expression and print." Every time you run `grep` you are, quite literally, evaluating $L(R)$ for some regular expression R.

Worked examples: ten solved regular expressions

Unless stated otherwise, $\Sigma = \{a, b\}$.

1. *Only a's, any number, including none.* **Solution:** a^*.
2. *Strings starting with a.* **Solution:** $a(a|b)^*$.
3. *Strings ending in b.* **Solution:** $(a|b)^*b$.
4. *Strings containing aa somewhere.* **Solution:** $(a|b)^*aa(a|b)^*$.
5. *Some a's followed by some b's – Lecture 2's L_1L_2.* **Solution:** a^*b^*.
6. *ab repeated zero or more times.* **Solution:** $(ab)^*$.
7. *Over $\Sigma = \{0, 1\}$: strings ending in 0.* **Solution:** $(0|1)^*0$ – note this is a *superset* of Lecture 1's "binary encodings of even numbers": it also accepts strings with leading zeros, like 00 or 0100, which are not how Lecture 1's examples $(0, 10, 100, 110)$ were written. Restricting to canonical (no-leading-zero) numerals that are even needs $0 \mid 1(0|1)^*0$ instead.
8. *Over $\Sigma = \{0, 1\}$: binary numerals with no leading zero (except "0" itself).* **Solution:** $0 \mid 1(0|1)^*$.
9. *Identifiers: a letter followed by any number of letters or digits – Lecture 2's motivating example, finally solved.* **Solution:** writing L for "any letter" and D for "any digit," $L(L|D)^*$.
10. *Strings with an* even *number of a's (zero counts as even).* **Solution:** $(b^*ab^*ab^*)^*$ – each pass through the star consumes exactly two a's, so the total is always even.

Exercises: Writing Regular Expressions

Unless stated otherwise, $\Sigma = \{a, b\}$.

1. Write a regular expression for strings containing the substring bb. (Hint: mirror solved example 4.)
2. Over $\Sigma = \{0, 1\}$, write a regular expression for strings with an *odd* number of 1's, and compare it against solved example 10's regex for an even number of a's – how are the two related?
3. Write a regular expression for strings that start and end with the *same* symbol.

1.3 Grammars: A Different Way to Generate L

Regular expressions generate a language by combining whole sub-languages with union, concatenation, and star. A **grammar** generates a language differently: by rewriting symbols, one substitution at a time, starting from a single designated symbol.

Grammar

A **grammar** is a 4-tuple $G = (V, \Sigma, P, S)$:

- V – a finite set of **variables** (non-terminals): placeholders that never appear in a finished string.
- Σ – a finite set of **terminals**, the alphabet of the language being generated, with $V \cap \Sigma = \emptyset$.
- P – a finite set of **production rules**, each of the form $\alpha \to \beta$ with $\alpha \in (V \cup \Sigma)^* V (V \cup \Sigma)^*$ (a string of variables and/or terminals, containing at least one variable) and $\beta \in (V \cup \Sigma)^*$.
- $S \in V$ – the **start symbol**, where every generation begins.

That generality in α – a whole string, not just one variable – is exactly what gives context-sensitive and unrestricted grammars their extra power later in this course: a rule can rewrite a variable differently depending on what surrounds it. Every grammar in this document restricts α to a single variable, $A \in V$; that restricted case is called a **context-free grammar** (rules $A \to \beta$), and it is what we build with from here on.

We stop here, at the shape of the rules themselves. How a grammar actually turns S into a finished string – *derivation* – is forthcoming lecture's question; for now, just examples of the tuple.

Worked example: matched a's and b's

$V = \{S\}$, $\Sigma = \{a, b\}$, start symbol S, and

$$P = \{\, S \to aSb \mid \varepsilon \,\}.$$

Strings like ε, ab, $aabb$, $aaabbb$ are meant to come from this grammar – we will make precise *how* in the future lecture.

Worked example: balanced parentheses

$V = \{S\}$, $\Sigma = \{(,\ )\}$, start symbol S, and

$$P = \{\, S \to (S)S \mid \varepsilon \,\}.$$

Worked example: simple arithmetic expressions

$V = \{E\}$, $\Sigma = \{a, +, *, (,)\}$, start symbol E, and

$$P = \{\, E \to E + E \mid E * E \mid (E) \mid a \,\}.$$

Notice a single variable can have several right-hand sides, separated by $|$ – shorthand for several separate rules sharing the same left-hand side. This grammar is, *on purpose*, **ambiguous**: a string like $a + a * a$ has more than one valid parse, since nothing in the rules says whether $+$ or $*$ groups first – exactly the dangling-else disease from Lecture 1, now showing up in a grammar instead of a compiler rule. We leave it broken here deliberately; fixing it (by encoding precedence into the rules, or by other means) is standard next-lecture material.

Exercises: Grammars

1. For the matched-a's-and-b's grammar above, write out V, Σ, P, and S as four separate, explicitly labelled items.
2. Write production rules P (with $V = \{S\}$, $\Sigma = \{a, b\}$, start symbol S) that you would guess generate *any* number of a's followed by *any* number of b's, independently – then compare against the regex you would write for the same language (solved example 5).
3. Write a grammar (V, Σ, P, S) whose rules are meant to generate $\{\varepsilon, aa, aaaa, aaaaaa, \dots\}$ – strings of an even number of a's – and compare it against solved example 10's regex for the same language.

2 References

1. (SIPSER) Michael Sipser, *Introduction to the Theory of Computation*, 3rd ed., Cengage Learning, 2012.
2. (HMU) John E. Hopcroft, Rajeev Motwani, Jeffrey D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3rd ed., Pearson, 2006.

3 Further Readings

1. (HMU) §3.1 "Regular Expressions" (3.1.1 The Operators of Regular Expressions, 3.1.2 Building Regular Expressions) – HMU's own treatment of the recursive definition built in this document.
2. (SIPSER) §1.3 "Regular Expressions" – Sipser's version of the same definition, followed there by the proof that regular expressions and finite automata define the same class of languages.

3. (HMU) §5.1 "Context-Free Grammars" (5.1.2 Definition of Context-Free Grammars) – HMU's tuple definition of a grammar, matching this document's stopping point at production rules.

4. (SIPSER) §2.1 "Context-Free Grammars" – Sipser's version of the same tuple, going on (beyond where we stop here) to define derivations and parse trees.

4 Disclaimer

The above document is intended to be a reading material/class note for an undergraduate course on Theory of Computation. This first draft may contain errors as it has not been reviewed. Feel free to let me know the gaps, errors, and typos you encounter in the document.

1 Recognizers, Regular Expressions, and Grammars

Lecture 1 ended on an open question: given only Σ and a language as a subset of Σ^*, how do we *finitely* specify which strings are in it? Lecture 2 built the algebra – Σ^*, concatenation, Kleene star, closure – that any answer would need. Now we answer it: there are exactly two dual ways to specify a language, **recognizers** and **generators**. We define both, then build out the generator side with two concrete tools: **regular expressions**, and **grammars** – stopping right at a grammar's production rules, before derivations.

1.1 The Specification Problem: Two Dual Answers

A language L can be pinned down in exactly two dual ways: build a machine that reads a string and answers yes or no, or build a device that writes out exactly the strings of L, without reading any input at all.

Recognizers and generators

- A **recognizer** (an **automaton**) is a machine M that takes a string w as input and outputs ACCEPT or REJECT. It defines the language $L(M) = \{w : M \text{ accepts } w\}$.
- A **generator** (a **grammar** or **regular expression**) is a device G that produces strings by rule, with no input to read. It defines the language $L(G) = \{w : G \text{ can generate } w\}$.

This course studies both halves. The rest of this document builds the generator side.

1.2 Regular Expressions: A Finite Notation for $L(R)$

Recall Lecture 2's algebra: concatenation, union, and Kleene star build new languages out of old ones. A **regular expression** is a finite string of symbols that names one specific language, built out of exactly those three operations, recursively.

Recursive definition of a regular expression over Σ

Base cases:

- $\emptyset$ is a regular expression, $L(\emptyset) = \emptyset$.
- ε is a regular expression, $L(\varepsilon) = \{\varepsilon\}$.
- For each $a \in \Sigma$, a is a regular expression, $L(a) = \{a\}$.

Inductive cases: if R_1, R_2 are regular expressions, so are

- $(R_1|R_2)$, with $L(R_1|R_2) = L(R_1) \cup L(R_2)$;
- (R_1R_2), with $L(R_1R_2) = L(R_1)L(R_2)$;
- $(R_1)^*$, with $L((R_1)^*) = L(R_1)^*$.

Nothing else is a regular expression.

Every piece of that definition already has a name from Lecture 2 – union, concatenation, Kleene star – just written as compact linear notation instead of set-builder notation. A regular expression *is* a recipe for building a language the same way we built L_1L_2 and L^* before, except now the recipe itself is a string you can write on one line.

grep, and where the notation actually earns its keep

Ken Thompson built regular-expression matching into the `QED` and `ed` text editors in the late 1960s, using exactly Kleene's 1951 closure notation (Lecture 2). The Unix command `grep` gets its name from the `ed` command `g/re/p` – "globally search for a regular expression and print." Every time you run `grep` you are, quite literally, evaluating $L(R)$ for some regular expression R.

1.3 Worked Examples: Ten Solved Regular Expressions

Unless stated otherwise, $\Sigma = \{a, b\}$.

1. *Only a's, any number, including none.* **Solution:** a^*.
2. *Strings starting with a.* **Solution:** $a(a|b)^*$.
3. *Strings ending in b.* **Solution:** $(a|b)^*b$.
4. *Strings containing aa somewhere.* **Solution:** $(a|b)^*aa(a|b)^*$.
5. *Some a's followed by some b's – Lecture 2's L_1L_2.* **Solution:** a^*b^*.
6. *ab repeated zero or more times.* **Solution:** $(ab)^*$.
7. *Over $\Sigma = \{0, 1\}$: strings ending in 0.* **Solution:** $(0|1)^*0$ – note this is a *superset* of Lecture 1's "binary encodings of even numbers": it also accepts strings with leading

zeros, like 00 or 0100, which are not how Lecture 1's examples $(0, 10, 100, 110)$ were written. Restricting to canonical (no-leading-zero) numerals that are even needs $0 \mid 1(0|1)^*0$ instead.

8. *Over $\Sigma = \{0, 1\}$: binary numerals with no leading zero (except "0" itself).* **Solution:** $0 \mid 1(0|1)^*$.

9. *Identifiers: a letter followed by any number of letters or digits – Lecture 2's motivating example, finally solved.* **Solution:** writing L for "any letter" and D for "any digit," $L(L|D)^*$.

10. *Strings with an* even *number of a's (zero counts as even).* **Solution:** $(b^*ab^*ab^*)^*$ – each pass through the star consumes exactly two a's, so the total is always even.

Exercises: Writing Regular Expressions

Unless stated otherwise, $\Sigma = \{a, b\}$.

1. Write a regular expression for strings containing the substring bb. (Hint: mirror solved example 4.)

2. Over $\Sigma = \{0, 1\}$, write a regular expression for strings with an *odd* number of 1's, and compare it against solved example 10's regex for an even number of a's – how are the two related?

3. Write a regular expression for strings that start and end with the *same* symbol.

1.4 Grammars: A Different Way to Generate L

Regular expressions generate a language by combining whole sub-languages with union, concatenation, and star. A **grammar** generates a language differently: by rewriting symbols, one substitution at a time, starting from a single designated symbol.

Grammar

A **grammar** is a 4-tuple $G = (V, \Sigma, P, S)$:

- V – a finite set of **variables** (non-terminals): placeholders that never appear in a finished string.
- Σ – a finite set of **terminals**, the alphabet of the language being generated, with $V \cap \Sigma = \emptyset$.
- P – a finite set of **production rules**, each of the form $\alpha \to \beta$ with $\alpha \in (V \cup \Sigma)^* V (V \cup \Sigma)^*$ (a string of variables and/or terminals, containing at least one variable) and $\beta \in (V \cup \Sigma)^*$.
- $S \in V$ – the **start symbol**, where every generation begins.

That generality in α – a whole string, not just one variable – is exactly what gives context-sensitive and unrestricted grammars their extra power later in this course: a rule can rewrite a variable differently depending on what surrounds it. Every grammar in this document restricts α to a single variable, $A \in V$; that restricted case is called a **context-free grammar** (rules $A \to \beta$), and it is what we build with from here on.

We stop here, at the shape of the rules themselves. How a grammar actually turns S into a finished string – *derivation* – is forthcoming lecture's question; for now, just examples of the tuple.

Worked example: matched a's and b's

$V = \{S\}$, $\Sigma = \{a, b\}$, start symbol S, and

$$P = \{\, S \to aSb \mid \varepsilon \,\}.$$

Strings like ε, ab, $aabb$, $aaabbb$ are meant to come from this grammar – we will make precise *how* in the future lecture.

Worked example: balanced parentheses

$V = \{S\}$, $\Sigma = \{(,\ )\}$, start symbol S, and

$$P = \{\, S \to (S)S \mid \varepsilon \,\}.$$

Worked example: simple arithmetic expressions

$V = \{E\}$, $\Sigma = \{a, +, *, (,)\}$, start symbol E, and

$$P = \{\, E \to E + E \mid E * E \mid (E) \mid a \,\}.$$

Notice a single variable can have several right-hand sides, separated by $\mid$ – shorthand for several separate rules sharing the same left-hand side. This grammar is, *on purpose*, **ambiguous**: a string like $a + a * a$ has more than one valid parse, since nothing in the rules says whether $+$ or $*$ groups first – exactly the dangling-else disease from Lecture 1, now showing up in a grammar instead of a compiler rule. We leave it broken here deliberately; fixing it (by encoding precedence into the rules, or by other means) is standard next-lecture material.

Exercises: Grammars

1. For the matched-a's-and-b's grammar above, write out V, Σ, P, and S as four separate, explicitly labelled items.
2. Write production rules P (with $V = \{S\}$, $\Sigma = \{a, b\}$, start symbol S) that you would guess generate *any* number of a's followed by *any* number of b's, independently – then compare against the regex you would write for the same language (solved example 5).
3. Write a grammar (V, Σ, P, S) whose rules are meant to generate $\{\varepsilon, aa, aaaa, aaaaaa, \dots\}$ – strings of an even number of a's – and compare it against solved example 10's regex for the same language.

2 References

1. (SIPSER) Michael Sipser, *Introduction to the Theory of Computation*, 3rd ed., Cengage Learning, 2012.
2. (HMU) John E. Hopcroft, Rajeev Motwani, Jeffrey D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3rd ed., Pearson, 2006.

3 Further Readings

1. (HMU) §3.1 "Regular Expressions" (3.1.1 The Operators of Regular Expressions, 3.1.2 Building Regular Expressions) – HMU's own treatment of the recursive definition built in this document.
2. (SIPSER) §1.3 "Regular Expressions" – Sipser's version of the same definition, followed there by the proof that regular expressions and finite automata define the same class of languages.
3. (HMU) §5.1 "Context-Free Grammars" (5.1.2 Definition of Context-Free Grammars) – HMU's tuple definition of a grammar, matching this document's stopping point at production rules.
4. (SIPSER) §2.1 "Context-Free Grammars" – Sipser's version of the same tuple, going on (beyond where we stop here) to define derivations and parse trees.

4 Disclaimer

The above document is intended to be a reading material/class note for an undergraduate course on Theory of Computation. This first draft may contain errors as it has not been reviewed. Feel free to let me know the gaps, errors, and typos you encounter in the document.